SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

LoanAI is an intelligent multi-agent loan processing system designed specifically for the Malaysian banking sector. The solution addresses the traditionally slow and manual process of loan application and assessment by leveraging an AI-powered pipeline to transform unstructured customer inputs into structured, data-driven loan decisions.

1.1. Purpose:

This system analysis document highlights the technical scope and design-related decisions behind the development of LoanAI. The key elements covered include:

1. ***Architecture**:*
    a. *A multi-agent orchestration framework where specialized agents handle specific stages of the loan lifecycle, ensuring modularity and scalability.*
2. ***Data Flows:***
    a. *Traces the journey of unstructured data (chat text and document uploads like MyKad, payslips, and bank statements) through extraction, validation, and analysis to a final decision.*
3. ***Model Process:***
    a. *Showcases the end-to-end workflow from initial user engagement via natural language to the final delivery of a credit verdict.*
4. *Role of Reference:*
    a. *This document serves as a blueprint for developers and stakeholders to understand the integration of multimodal AI within a regulated financial context.*

1.2. Background:

In the current Malaysian financial landscape, loan processing often involves significant manual intervention, leading to delays and potential human error. Customers find the documentation process cumbersome, and banks struggle with the volume of unstructured data.

1. *Previous Version:*
    a. *Traditional systems rely on manual data entry and rule-based engines that cannot interpret visual documents or conversational nuances.*
2. *Changes in Major Architectural Components: LoanAI introduces a "Multi-Agent Orchestrator" that replaces linear processing with dynamic task delegation.*
3. *New capabilities:*
    a. *Multimodal Extraction: Using Gemini 2.0 Flash to "read" Malaysian financial documents.*
    b. *Real-time Pipeline Visualization: Providing transparency to the user about the internal decision-making process.*
    c. *Automated DSR Calculation: Instant financial health assessment based on Bank Negara Malaysia (BNM) standards.*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
End Customer	*Applicants seeking personal or property loans.*	*Fast response times, minimal manual entry, and clear explanation of decisions.*
Loan Officer	*Reviewing and auditing AI-generated decisions.*	*Accurate data extraction, comprehensive audit trails (Agent Trailing), and risk highlights.*
Bank Admin	*Managing system configurations and API keys.*	*Secure handling of PII (Personally Identifiable Information) and system reliability.*
Development Team	*Building and maintaining the agentic pipeline.*	*Modular code, clear API contracts between agents, and efficient LLM token usage.*
Compliance / QA	*Ensuring adherence to BNM guidelines.*	*Transparent DSR calculations and adherence to e-KYC standards.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Intelligent Agentic Platform*
Architecture	*Multi-Agent Orchestration with Server-side API Integration*

LoanAI is structured as a multi-agent system where a central Orchestrator Agent manages the lifecycle of a loan application. The system is designed to be "AI-native," meaning every core function from data entry to risk assessment is powered by specialized LLM agents.

2.1.2 LLM as Service Layer

The system integrates Gemini 2.0 Flash as the core reasoning and multimodal engine. Unlike generic "AI modules," Gemini is used as a "Reasoning-in-the-Loop" service that:

- Performs OCR and semantic understanding of Malaysian documents (MyKad, EPF statements).
- Conducts multi-step financial reasoning for DSR calculations.
- Generates natural language clarifications for missing data.

2.1.3 Dependency Diagram

The following logic governs the agent interactions: a. Prompt Construction: Prompts are dynamically built using system instructions tailored for each agent (e.g., "You are a Credit Check Agent…"). b. Context Window: The Orchestrator maintains a shared "Application State" that is passed between agents, ensuring continuity. c. Parsing: Outputs from agents are strictly parsed into JSON formats for downstream consumption. d. Token Management: Gemini 2.0 Flash's efficiency is leveraged to handle large document contexts without exceeding rate limits. e. API Calls: Secure server-side calls via .env persisted keys to Gemini API and simulated banking databases. f. Interactions: The Frontend (React/Vite) communicates with the Orchestrator via a FastAPI backend, which then triggers the agent pipeline.

© UMHackathon 2026

2.2. Technological Stack

Layer	*Technology*	*Justification*
Frontend	*React + Tailwind CSS*	*Fast, responsive UI for real-time pipeline visualization and interactive gauges.*
Backend	*Python (FastAPI)*	*Asynchronous support for long-running agentic tasks and seamless integration with AI SDKs.*
AI Engine	*Gemini 2.0 Flash*	*Multimodal capabilities for document extraction and low-latency reasoning.*
Orchestration	*Custom Agentic Framework*	*Tailored multi-agent coordination for specific loan processing workflows*
Security	*Dotenv (.env)*	*Server-side API key persistence to prevent exposure of sensitive credentials.*

2.3. Key Data Flows

2.3.1. The data flow within LoanAI follows a "Collect -> Extract -> Analyze -> Decide" sequence:

2.3.1.1. Data Flow Diagram (DFD)

a. *User Input: Customer uploads files (PDF/JPG) or types messages in the Chat Interface.*
b. *Extraction: The Extraction Agent processes the files, identifying MyKad details, monthly income from payslips, and commitments from bank statements.*
c. *Validation: The Validation Agent checks for completeness. If a payslip is missing, it prompts the user via the chat.*
d. *Analysis: The Financial Analysis Agent calculates the Debt Service Ratio (DSR) using the formula: (Total Monthly Commitments / Net Monthly Income) x 100.*
e. *Decision: The Decision Agent cross-references the DSR and credit score (simulated) against bank thresholds to deliver a verdict.*

2.3.2. Normalized Database Schema

The database is designed to 3NF to ensure integrity:

- Users: Basic profile and authentication.
- Applications: Linked to Users, stores the current status (Pending, Approved, Rejected). Documents: Metadata of uploaded files linked to an Application.
- Agent_Logs: Stores "Agent Trailing" data—what each agent did, when, and their output for auditability.
- Financial_Profiles: Structured data extracted from documents (Income, Expenses, DSR).

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	**Natural Language Chat**	A conversational interface that guides the user through the application process instead of static forms.
2	**Multimodal Extraction**	Automatic data extraction from MyKad (IC), Payslips, and Bank Statements using Gemini 2.0 Flash.
3	**Real-time Pipeline Viz**	A visual UI component showing which agent (Orchestrator, Extraction, etc.) is currently active.
4	**Interactive DSR Gauge**	A dynamic dashboard element that calculates and displays the user's DSR in real-time.
5	**Agent Trailing**	A detailed log of each agent's contribution to the decision, visible for audit purposes.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	Handle multiple concurrent loan applications.	Use of FastAPI's async capabilities and stateless agent logic.
Security	Protect sensitive financial data and PII.	Server-side processing, no storage of raw files after extraction (optional), and .env for keys.
Reliability	99% accuracy in data extraction from clear documents.	Prompt engineering with few-shot examples for Gemini 2.0 Flash.
Latency	Decision pipeline completion under 30 seconds.	Parallel processing of documents where possible; optimized LLM prompts.
Auditability	Every decision must be traceable.	Implementation of the "Agent Trailing" feature recording all LLM interactions.

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

a. ***Direct CCRIS/CTOS Integration: Currently simulated for the hackathon.***

b. ***Real-time Property Valuation: Currently uses a simplified valuation agent.***

c. ***Automated Disbursement: Direct integration with bank core systems for fund transfer.***

d. ***Multi-language Support: Expanding beyond English to include Bahasa Melayu and Mandarin. —***

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: For a production deployment, LoanAI would initially be released to a small segment of bank employees (internal testing) before a phased rollout to 5% of the customer base. A/B testing would compare the "AI-Agent" flow against the traditional "Form-based" flow to measure conversion rates and time-to-decision.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: In case of systematic miscalculations (e.g., incorrect DSR logic), the system can trigger an Emergency Rollback to a "Golden Release"—a stable version with manual review overrides for all AI decisions.*

4.1.3. Priority Matrix:

4.1.3.1. ***P1 (Critical):*** *Data extraction failure or incorrect DSR calculation. Action: Immediate rollback and manual queue redirection.*

4.1.3.2. ***P2 (High):*** *Latency exceeding 60 seconds. Action: Optimize prompt length or switch to a faster model tier.*

4.1.3.3. ***P3 (Medium):*** *UI glitches in the pipeline visualization. Action: Patch in next sprint.*

© UMHackathon 2026

*4.2. **Monitoring**:*

LoanAI utilizes a centralized dashboard to monitor:

- *Agent Health: Success/Failure rate of each agent in the pipeline.*
- *Token Usage: Real-time tracking of Gemini API costs.*
- *Decision Accuracy: Comparing AI decisions against a "ground truth" set of manually reviewed applications.*

4.3. Assumptions:

a. Users have access to clear digital copies or photos of their financial documents.

b. The Gemini 2.0 Flash API maintains stable uptime during the hackathon.

c. Malaysian document formats (e.g., standard payslips) follow a semi-structured pattern recognizable by multimodal AI.

*4.4. **External Dependencies:***

Tools	Purpose	Risks
Gemini 2.0 Flash API	Core LLM reasoning and multimodal extraction.	Rate limits or API downtime.
FastAPI	Backend framework.	Breaking changes in future updates.
Vite/React	Frontend development.	Dependency conflicts.
Dotenv	Secret management.	Misconfiguration of environment variables.

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Project Management & Team Contributions

The project development is structured over a intensive hackathon period (approx. 48- 72 hours).

5.1. Project Timeline:

5.1.1. *Day 1: System Design, Schema Setup, and Prompt Engineering for Extraction/Validation Agents. Day 2: Backend Integration, Frontend UI Development (Chat & Gauges), and Agent Orchestration. Day 3: Testing, Agent Trailing implementation, and Final Presentation prep.*

5.2. Team Members Role:

5.2.1. *Product Manager: PRD creation, workflow design, and compliance check.*

5.2.2. *Frontend Developer: UI/UX, Pipeline Visualization, and React integration.*

5.2.3. *Backend/AI Engineer: Agentic pipeline development, Gemini API integration, and DSR logic.*

5.2.4. *Data/QA Engineer: Document dataset preparation and decision validation.*

5.3. Recommendations:

5.3.1. *Scalability: Implement a message queue (e.g., Redis) if the volume of document processing increases.*

5.3.2. *Performance: Use prompt caching to reduce latency for repetitive validation tasks.*

5.3.3. *Reliability: Introduce a human-in-the-loop (HITL) step for "Conditional" approvals to ensure high-stakes decisions are verified.*

© UMHackathon 2026

Email: umhackathon@um.edu.my